COS 226 - Practical 4

Lock Granularity Experiment Results

Group members:
U24943802
U25135742
U25025822

Results (average execution time in ms)

Threads	Coarse-Grained Time	Fine-Grained Time
2	99.10	253.62
4	413.40	741.62
8	1532.80	1849.45
16	4908.67	3990.41

Individual runs (ms)

Run	2T Coarse	2T Fine	4T Coarse	4T Fine	8T Coarse	8T Fine	16T Coarse	16T Fine
1	106.68	277.36	441.69	675.33	1555.52	1809.96	5066.27	3990.09
2	87.95	219.05	401.35	890.54	1542.44	1833.25	4781.34	4120.42
3	126.25	278.53	441.93	774.74	1593.27	1838.57	4929.97	3965.62
4	103.55	290.87	413.25	696.51	1499.01	1849.38	4942.20	4079.98
5	71.08	202.29	368.76	670.97	1473.76	1916.09	4823.59	3795.93

Discussion

At low thread counts (2 and 4) the coarse-grained list is faster. The fine-grained list has to lock and unlock every node it passes while traversing, whereas the coarse-grained list takes one lock per operation. With few threads there is little contention.

As the thread count increases, the coarse-grained time grows quickly. Only one thread can be inside the list at a time, so all operations are serialised and threads spend more and more time waiting for the single lock. The list also gets longer with more threads (each thread adds its own values), so every operation takes longer while holding that lock.

The fine-grained time grows more slowly (roughly 2-3x each time the threads double), because with hand-over-hand locking several threads can be inside the list at the same time, each holding locks on different nodes. The gap closes at 8 threads and at 16 threads the fine-grained list is faster.

This shows that the fine-grained list scales better, its performance degrades less as more threads are added. It does not scale perfectly, because every thread must first lock the head node, and threads cannot overtake each other while traversing, so they queue behind one another along the list. The trade-off is lower overhead for coarse-grained locking at low contention versus more concurrency for fine-grained locking at high contention.